FixFirst

Find the cause first, then prove the fix

IRS Practice Module · Project Proposal Presentation · Group 24

HE ZIHANG (A0xxxx72R) · WEI YI (A0xxxx64H) · CHI YONGJUN (A0xxxx66A)

September 2026

What FixFirst will do

Similar errors can have different causes. FixFirst will use evidence to find the cause, show why, and only call a problem fixed when a real check passes.



1 Check

Run the project's own checks: environment, dependencies, pytest, Ruff

2 Diagnose

Find the root cause of each failure and rank what to do next

3 Verify

Close a problem only when a real check passes

A local tool for small Python projects · the user makes the changes · no cloud model needed

When Python projects break

1

Taking over a project

A teammate's code, a course assignment or a thesis tool

2

Upgrading Python

3.12 removed `distutils` and `imp`; 3.13 removed 19 old standard-library modules

3

Upgrading a library

NumPy 2.0 and SQLAlchemy 2.0 removed or changed names that older code relies on

4

Reproducing results

Research code, tutorials and projects pinned to old versions

Who meets this: students, junior developers, maintainers of small projects and researchers

WHY IT MATTERS

1,921 of 2,702

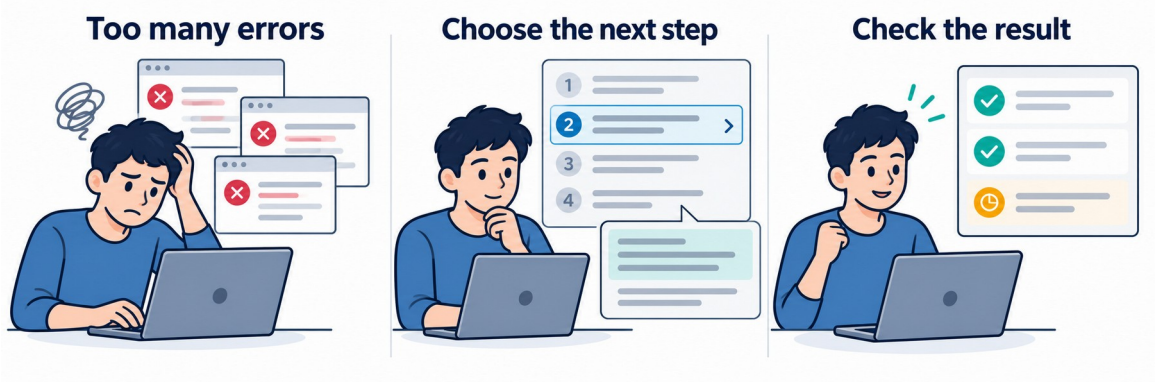
historical Python builds no longer reproduced because of dependency errors

Source: Mukherjee et al., PyDFix, ISSTA 2021 — across the builds that study examined

Python and library upgrades can expose compatibility problems.

Scenario: same error type, different causes

Wang takes over a teammate's Python project before a deadline. The tests fail with dozens of messages, and he cannot tell where to start.



Error	Real cause	Fix
`No module named 'requests'`	A declared package is not installed	Install it
`No module named 'helper'`	The project file was renamed to `helpers.py`	Fix the import
`No module named 'imp'`	Python 3.12 removed `imp`	Change the code or use an older Python

The error type is the same; evidence about the project and its environment tells the causes apart.

Three questions FixFirst will answer

1 Which cause?

The same error can have different causes

TODAY

pip, pytest and Ruff report the error, not the cause

FIXFIRST WILL

Find the root cause from evidence and knowledge, and show the rule and source

Fix the real cause

2 What first?

Results are scattered and unordered

TODAY

Each tool reports its own part, in output order

FIXFIRST WILL

Run the checks together and order the steps by the goal, blockers first

No time on downstream symptoms

3 Did it work?

A change can look like a fix

TODAY

A partial rerun can look like success

FIXFIRST WILL

Close a problem only when a check of the same scope passes

Honest progress

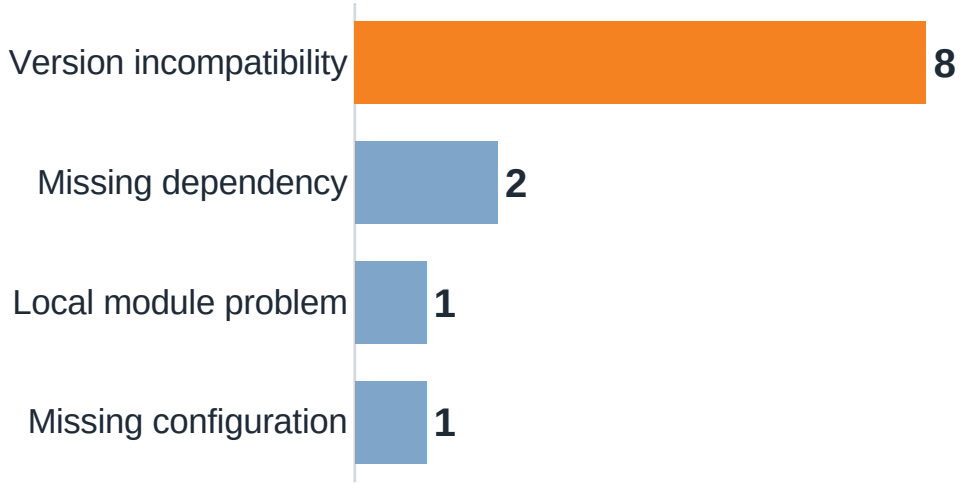
Coding assistants can also run tests; FixFirst differs by design: it runs locally, shows its evidence and records what each check covered.

Data we can use, and what it tells us

Sources

Sources	DONE	TO ADD
Generated fault projects Train and test; ≥ 30 per cause	**215** cases run	**Hard cases** as a separate set
Real open-source projects Realistic held-out test	**13** pilot projects	**+10–15** new projects
Real library defects Verify fixes; more from BugsInPy	**4** in 2 libraries	**+4** for ≥ 8 in 4 libraries
PyDFix Small external text check	**22** cases collected	**Labels** for all 22

Pilot survey: causes of 12 failing projects



13 open-source projects installed with today's libraries: 12 failed; 3 involved behaviour changes with no clear error. The configuration case: an environment that was not activated.

So we will keep the five causes balanced (≥ 30 cases each), sample the new real projects by cause, report per-class results and Macro-F1, and keep hard cases apart.

What the system needs from the data

ONE FAILING TEST

- Error message
- Traceback
- Environment snapshot
- Project files

EVIDENCE

The error itself
Exception type, stage, key phrases

The traceback
Where it was raised

Environment and project
Installed? Standard library? Local file? Declared?

FEATURES (~40 YES/NO)

- **Exception type**
- **Failure stage**
- **Where it was raised**
- **Module status**
- **Text signals**

LABEL (ONE OF FIVE)

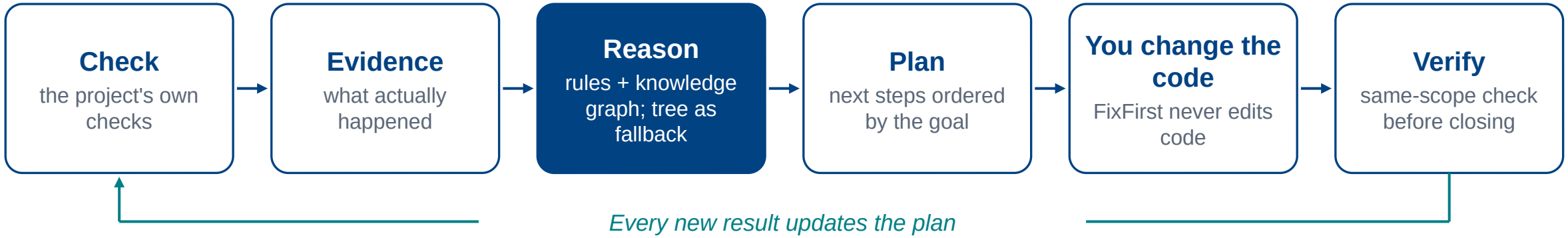
- Missing dependency
- Local module problem
- Version incompatibility
- Missing configuration
- Code defect

EXAMPLE `No module named 'requests'` → ModuleNotFoundError: yes · in project code: yes · installed: no

Labels generated projects: the injected fault · real projects: manually labelled and reviewed before FixFirst runs

Features never include the answer, such as FixFirst's own error category

Proposed architecture



DECISION AUTOMATION

Rules

Forward chaining connects evidence and knowledge to a cause and an action. It answers only when the conditions hold.

KNOWLEDGE REPRESENTATION

Knowledge graph

Facts from official release notes: what was removed, when, and what replaces it. Links every step to its evidence.

DATA MINING

Decision tree

Suggests a likely cause when no rule applies, shown as unconfirmed. TF-IDF similarity groups repeated messages.

- **Decision order:** rules with the knowledge graph first; the decision tree only when no rule applies, marked unconfirmed.

How one decision will be made

Worked example

1 Evidence

`pricing.py` line 1:

```
from collections import Mapping
```

fails on Python 3.12

2 Knowledge

`collections.Mapping` was removed in Python 3.10; import it from `collections.abc` (Python docs)

3 Rule

A name the code uses + the release that removed it + the installed version

→ **version incompatibility**

4 Action

Suggest the fix: import it from `collections.abc` and say how to confirm

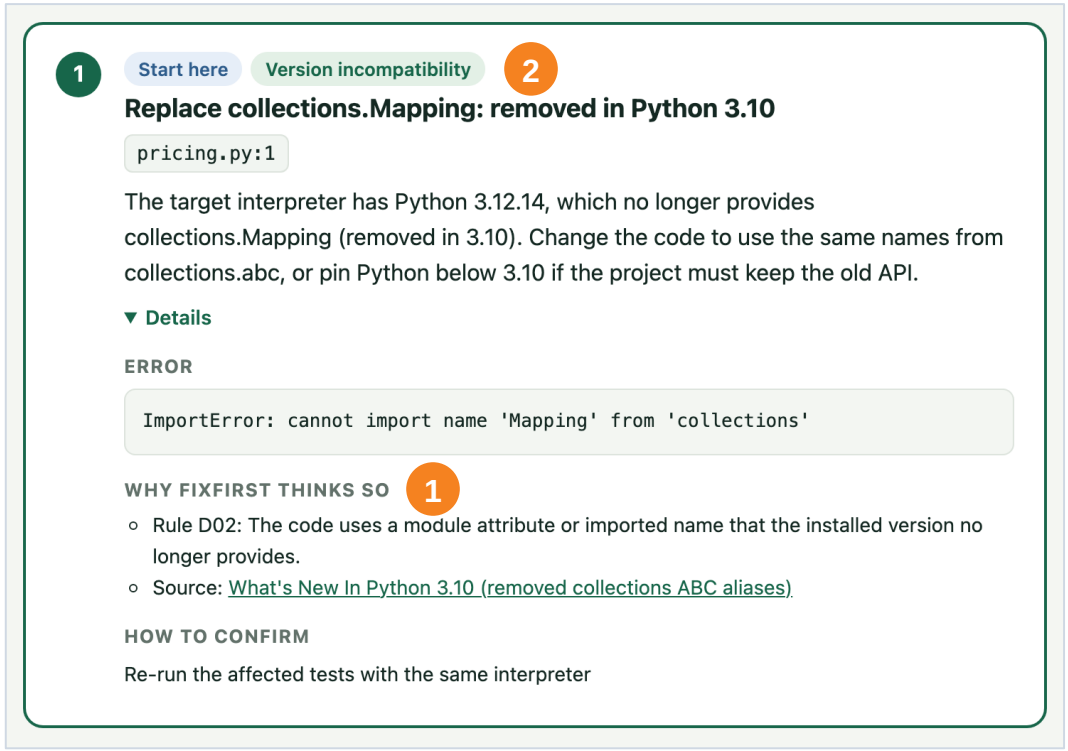
5 Verify

Close the problem only after a completed run covering it passes on the same interpreter

“The check tells us what failed, the knowledge graph supplies the version fact, and the rule connects them to a cause and a concrete action.”

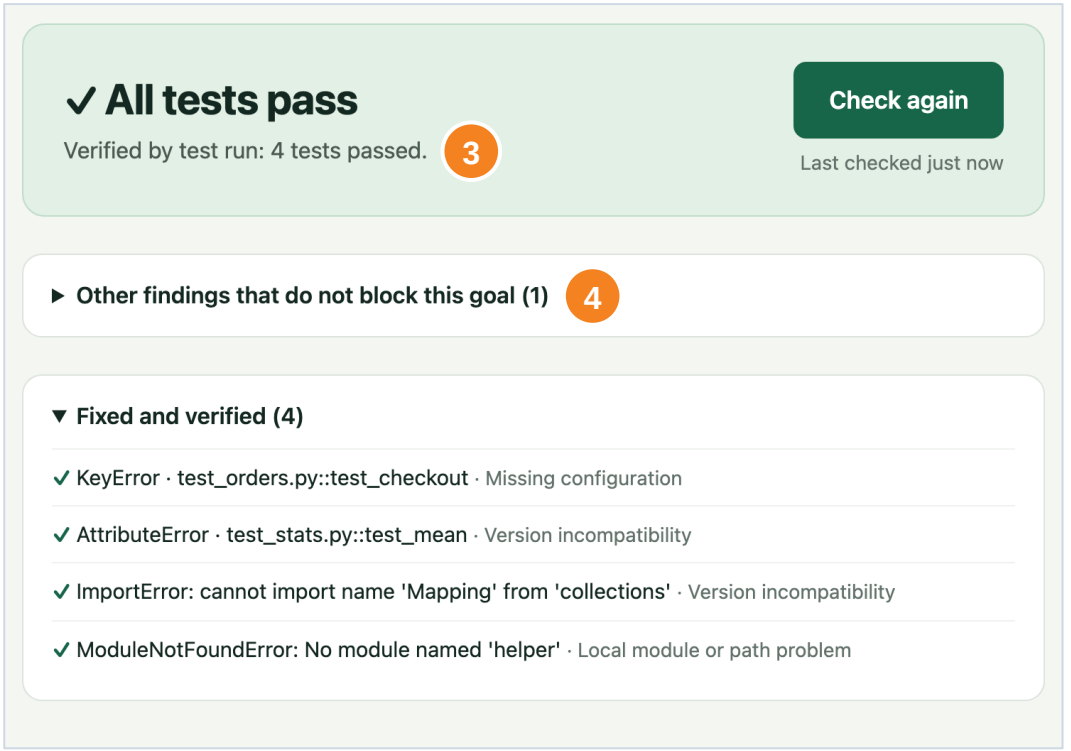
Expected output

What to do first, and why



The user edits the code, then presses Check again

After the user's changes: what is verified



1 Root cause and basis
Cause, file and line; the rule and its source

2 Ordered for the goal
What blocks the tests comes first

3 Honest verification
Closed only when a real check passes

4 Must fix vs optional
Findings that do not block the goal stay apart

Screens from our early prototype, on a sample project

Evaluation plan

1 Diagnosis

- Fault types held out from training
- Compared with error type alone, rules alone, tree alone
- Ablation: without the knowledge graph or the tree; Macro-F1

2 Next step and verification

- Compared with message order and a fixed category order
- No “fixed” claim without a covering check
- 10–15 new real projects labelled in advance; ≥ 8 real defects

3 User value

- 6–8 participants, with and without FixFirst; order balanced
- Both sides: terminal and web search only, no AI assistant
- Targets: 80% completion and 20% lower median time

Message grouping: precision, recall and F1 against exact matching, on repeats whose paths, line numbers and values differ

Labels come first · test data never tunes the rules · reviewers are not the rule authors

Main risks and how we will handle them

1	**RISK** **Our own rules may look better than they are**	**HOW WE WILL HANDLE IT** Hold out fault types, leave some faults outside the knowledge graph, and test on fresh real projects labelled by members who did not write the rules
2	**RISK** **Old environments are hard to rebuild**	**HOW WE WILL HANDLE IT** BugsInPy needs Python 3.6–3.8: start with the 70 candidates that need no compilation, and record the ones we cannot reproduce
3	**RISK** **Recruiting users takes time**	**HOW WE WILL HANDLE IT** Recruit now and run a 2–3 person pilot before the 6–8 person study

Scope: small Python projects; FixFirst does not edit code; behaviour changes that raise no clear error stay hard.

Goals and timeline

By 5 Oct

System and data

Lead: Reasoning, UI, data

DONE WHEN

Reasoning covers all five causes; 10–15 new real projects labelled before running

By 12 Oct

Evaluation

Lead: Data, reasoning

DONE WHEN

Held-out fault types, baselines and ablations; real projects; ≥ 8 real defects verified

By 19 Oct

User study

Lead: UI, data

DONE WHEN

6–8 participants; completion rate and time recorded

By 25 Oct

Deliverables

Lead: All members

DONE WHEN

Report, user guide, two videos, GitHub repository

We would welcome your feedback on the data plan and these priorities.

AI tool declaration

OpenAI Codex assisted with source research, translation, drafting and layout of the proposal, and an image generation tool produced its concept illustrations. OpenAI Codex and Anthropic Claude assisted with preparing this presentation. The interface screens on slide 10 are captured from our own prototype, not generated.

The team is responsible for all content.

Appendix A: evaluation details

Question	Compared with	Measure and target
Does it find the right root cause?	Error type alone; rules alone; decision tree alone	Accuracy and Macro-F1 on fault types held out from training
Does each technique help?	The full system without the knowledge graph, or without the tree	Change in accuracy (ablation)
Is the first step reasonable?	Message order; a fixed category order	Share of reasonable first steps; wasted attempts before the goal
Is verification honest?	Full runs, partial reruns, skipped tests	No “fixed” claim without a covering check
Does it work on real projects?	Labels written before running; new projects, not the 13 in the pilot	First step correct on 10–15 new held-out projects; ≥ 8 real defects in 4 libraries verified
Does it help users?	The same tasks without FixFirst; both sides use only the terminal and web search, no AI assistant; 6–8 participants; order balanced	80% completion; 20% lower median time
Are repeated messages grouped well?	Exact text matching; repeats with different paths, line numbers and values	Precision ≥ 0.90, reported with recall and F1

Appendix B: risks and limitations

Risk or limitation	How we will handle it
Generated faults are cleaner than real ones	Test on new real projects labelled before running; report the two separately
We write the rules and the knowledge, so results may be optimistic	Hold out fault types; leave some faults outside the knowledge graph on purpose; reviewers are not the rule authors
Public data is thin (22 usable PyDFix cases)	Use it as an external check only
Old environments are hard to rebuild (BugsInPy needs Python 3.6–3.8)	Start with the 70 bugs that need no compilation; record the ones we cannot reproduce
Recruiting 6–8 users takes time	Start recruiting now; run a 2–3 person pilot first
Scope	Small Python projects; behaviour changes that raise no error are hard to diagnose; FixFirst does not edit code